

АУТЕНТИФІКАЦІЯ ТА АВТОРИЗАЦІЯ. РОБОТА З WEB API. JSON WEB TOKEN.

Мета: набути навичок роботи з механізмами аутентифікації та авторизації користувачів в ASP.NET, набути навичок роботи з Web API.

Хід роботи:

Завдання 1.

Реалізувати реєстрацію, автентифікацію та авторизацію користувача з використанням пакету Microsoft Identity.

Після конфігурації Identity, реалізуйте наступні функціональності:

1. Реєстрація користувача ([приклад](#))
 - а. встановити вимогу по унікальності електронної адреси користувача
 - б. встановити вимоги до пароля: не коротший за 8 символів, містить цифри та літери, обов'язково містить хоча б одну літеру в верхньому регістрі
 - с. реалізувати поле для підтвердження введеного паролю
2. Автентифікація користувача (Login)
3. Вихід користувача (Logout)
4. Особистий кабінет користувача, де він матиме змогу переглядати або змінювати дані про себе.

Розробіть всі необхідні форми з підтримкою серверної валідації та розробіть відповідний контроллер. Налаштуйте доступ до Action-методів контроллерів з використанням атрибуту [Authorize] – лише залогінений користувач повинен мати можливість виконувати дії на веб-сайті, для неавтентифікованого користувача має бути доступною лише реєстрація та логін. Приклад коду контролера для виконання логіну:

```
using Microsoft.AspNetCore.Authorization;
```

Лістинг коду(Створення Реєстрації та Автентифікації користувачів):

```
using Microsoft.AspNetCore.Identity;  
using Microsoft.AspNetCore.Mvc;  
using Microsoft.IdentityModel.Tokens;  
using System.IdentityModel.Tokens.Jwt;  
using System.Security.Claims;  
using System.Text;
```

```
namespace CinemaBooking.Api.Controllers  
{  
    [Route("api/[controller]")]
```

					ДУ «Житомирська політехніка».26.121.08.000 – Лр5			
Змн.	Арк.	№ докум.	Підпис	Дата				
Розроб.		Грушевицький Д.І.			Звіт з лабораторної роботи	Літ.	Арк.	Аркушів
Перевір.		Українець М.О.					1	8
Керівник						ФІКТ Гр. ІПЗ-23-4		
Н. контр.								
Зав. каф.								

```

[ApiController]
public class AccountController : ControllerBase
{
    private UserManager<IdentityUser> userManager;
    private RoleManager<IdentityRole> roleManager;
    private IConfiguration configuration;

    public AccountController(UserManager<IdentityUser> userMgr,
        RoleManager<IdentityRole> roleMgr,
        IConfiguration config)
    {
        userManager = userMgr;
        roleManager = roleMgr;
        configuration = config;
    }

    [HttpPost("register")]
    public async Task<ActionResult> Register([FromBody] RegisterRequest model)
    {
        var user = new IdentityUser
        {
            UserName = model.UserName,
            Email = model.Email
        };

        var result = await userManager.CreateAsync(user, model.Password);
        if (result.Succeeded)
        {
            string role = model.Role ?? "User";
            if (await roleManager.RoleExistsAsync(role))
            {
                await userManager.AddToRoleAsync(user, role);
            }
            return Ok(new { Message = "Користувача зареєстровано успішно" });
        }

        return BadRequest(result.Errors);
    }

    [HttpPost("login")]
    public async Task<ActionResult> Login([FromBody] LoginRequest model)
    {
        var user = await userManager.FindByNameAsync(model.UserName);
        if (user != null && await userManager.CheckPasswordAsync(user, model.Password))
        {
            var roles = await userManager.GetRolesAsync(user);

            var claims = new List<Claim>
            {
                new Claim(ClaimTypes.Name, user.UserName!),
                new Claim(ClaimTypes.NameIdentifier, user.Id),
                new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
            };
        }
    }
}

```

		Грушевицький Д.І.			ДУ «Житомирська політехніка».26.121.08.000 – Лр5	Арк.
		Українець М.О.				
Змн.	Арк.	№ докум.	Підпис	Дата		2

```

        foreach (var role in roles)
        {
            claims.Add(new Claim(ClaimTypes.Role, role));
        }

        var key = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(configuration["Jwt:Key"]!));
        var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

        var token = new JwtSecurityToken(
            issuer: configuration["Jwt:Issuer"],
            audience: configuration["Jwt:Audience"],
            claims: claims,
            expires: DateTime.Now.AddHours(3),
            signingCredentials: creds
        );

        return Ok(new
        {
            Token = new JwtSecurityTokenHandler().WriteToken(token),
            Expiration = token.ValidTo
        });
    }

    return Unauthorized(new { Message = "Невірне ім'я або пароль" });
}

public class RegisterRequest
{
    public string Username { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
    public string? Role { get; set; }
}

public class LoginRequest
{
    public string Username { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
}
}

```

Лістинг коду(Program.cs):

```

using Microsoft.EntityFrameworkCore;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;
using CinemaBooking.Data;

```

```

var builder = WebApplication.CreateBuilder(args);

```

		Грушевицький Д.І.			ДУ «Житомирська політехніка».26.121.08.000 – Лр5	Арк.
		Українець М.О.				3
Змн.	Арк.	№ докум.	Підпис	Дата		

```

builder.Services.AddControllers()
    .AddJsonOptions(opts => {
        opts.JsonSerializerOptions.ReferenceHandler =
            System.Text.Json.Serialization.ReferenceHandler.IgnoreCycles;
    });

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddDbContext<CinemaDbContext>(opts => {
    opts.UseSqlite(builder.Configuration["ConnectionStrings:CinemaConnection"]);
});

builder.Services.AddDbContext<AppIdentityDbContext>(opts => {
    opts.UseSqlite(builder.Configuration["ConnectionStrings:IdentityConnection"]);
});

builder.Services.AddIdentity<IdentityUser, IdentityRole>(opts => {
    opts.Password.RequiredLength = 8;
    opts.Password.RequireDigit = true;
    opts.Password.RequireLowercase = true;
    opts.Password.RequireUppercase = true;
    opts.Password.RequireNonAlphanumeric = false;
    opts.User.RequireUniqueEmail = true;
}).AddEntityFrameworkStores<AppIdentityDbContext>();

builder.Services.AddAuthentication(opts => {
    opts.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    opts.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(opts => {
    opts.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = builder.Configuration["Jwt:Issuer"],
        ValidAudience = builder.Configuration["Jwt:Audience"],
        IssuerSigningKey = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!))
    };
});

builder.Services.AddScoped<ICinemaRepository, EFCinemaRepository>();
var app = builder.Build();
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
SeedData.EnsurePopulated(app);
IdentitySeedData.EnsurePopulated(app).Wait();

app.Run();

```

Лістинг коду(Ініціалізація адміна):

```
using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;

namespace CinemaBooking.Data
{
    public static class IdentitySeedData
    {
        public static async Task EnsurePopulated(IApplicationBuilder app)
        {
            using var scope = app.ApplicationServices.CreateScope();

            var context = scope.ServiceProvider.GetRequiredService<AppIdentityDbContext>();
            if (context.Database.GetPendingMigrations().Any())
            {
                context.Database.Migrate();
            }

            var roleManager = scope.ServiceProvider.GetRequiredService<RoleManager<IdentityRole>>();
            var userManager = scope.ServiceProvider.GetRequiredService<UserManager<IdentityUser>>();

            string[] roles = { "Admin", "User" };
            foreach (var role in roles)
            {
                if (!await roleManager.RoleExistsAsync(role))
                {
                    await roleManager.CreateAsync(new IdentityRole(role));
                }
            }

            string adminEmail = "admin@cinema.com";
            string adminPassword = "Admin1234";

            if (await userManager.FindByEmailAsync(adminEmail) == null)
            {
                var admin = new IdentityUser
                {
                    UserName = "admin",
                    Email = adminEmail
                };
                var result = await userManager.CreateAsync(admin, adminPassword);
                if (result.Succeeded)
                {
                    await userManager.AddToRoleAsync(admin, "Admin");
                }
            }
        }
    }
}
```

		Грушевицький Д.І.			ДУ «Житомирська політехніка».26.121.08.000 – Лр5	Арк.
		Українець М.О.				5
Змн.	Арк.	№ докум.	Підпис	Дата		

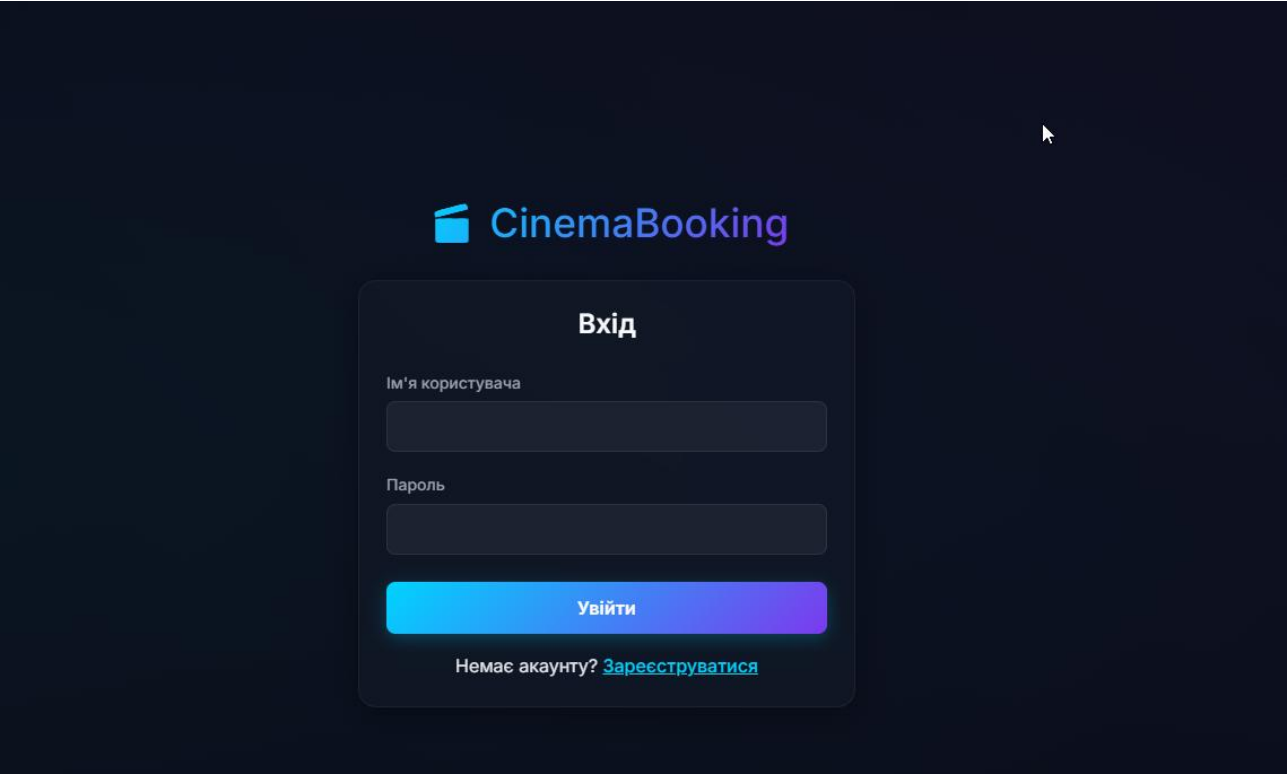


Рис. 1. Логін та реєстрація користувача

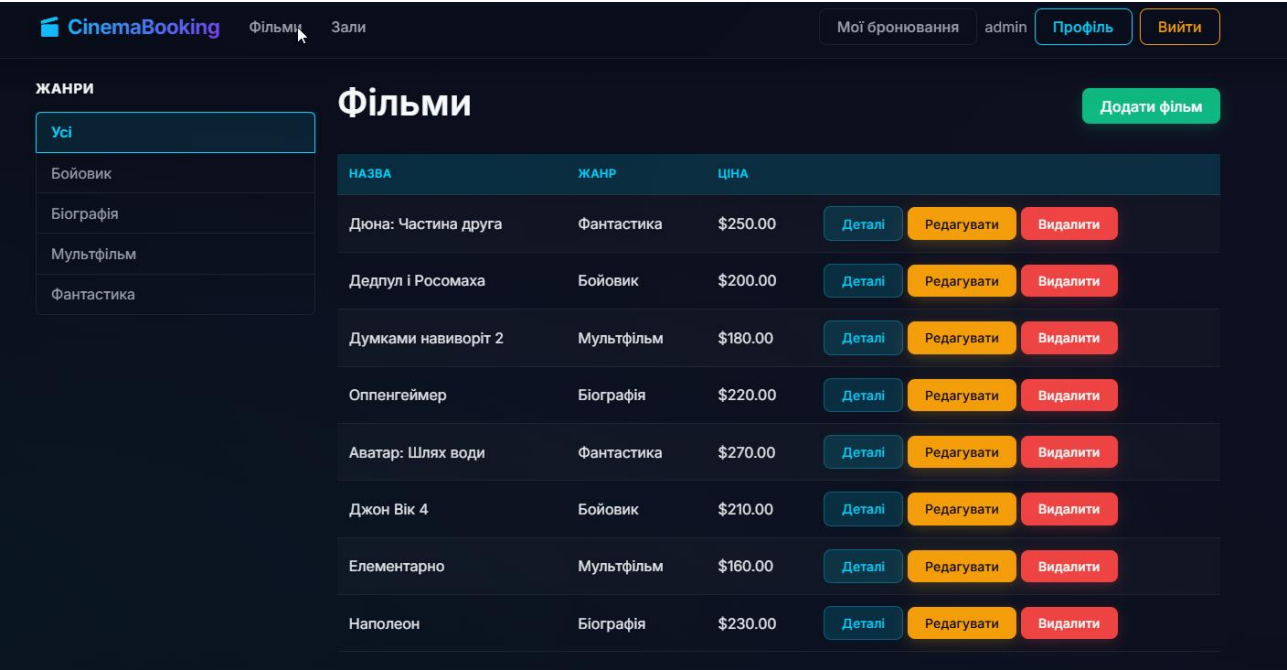


Рис. 2. Роль адміна

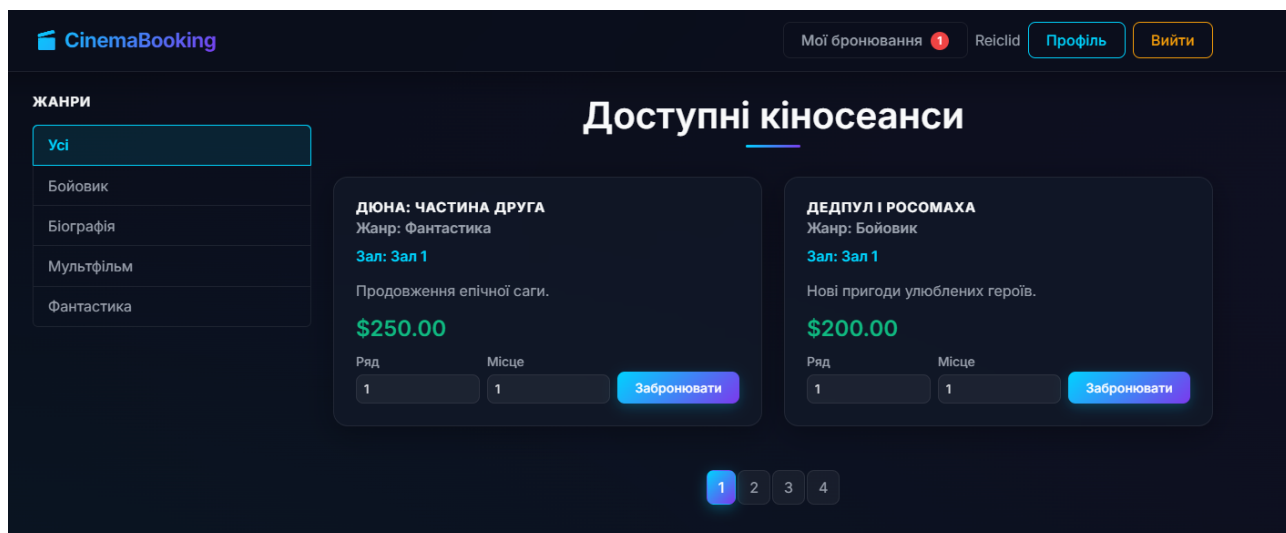


Рис. 4. Роль користувача

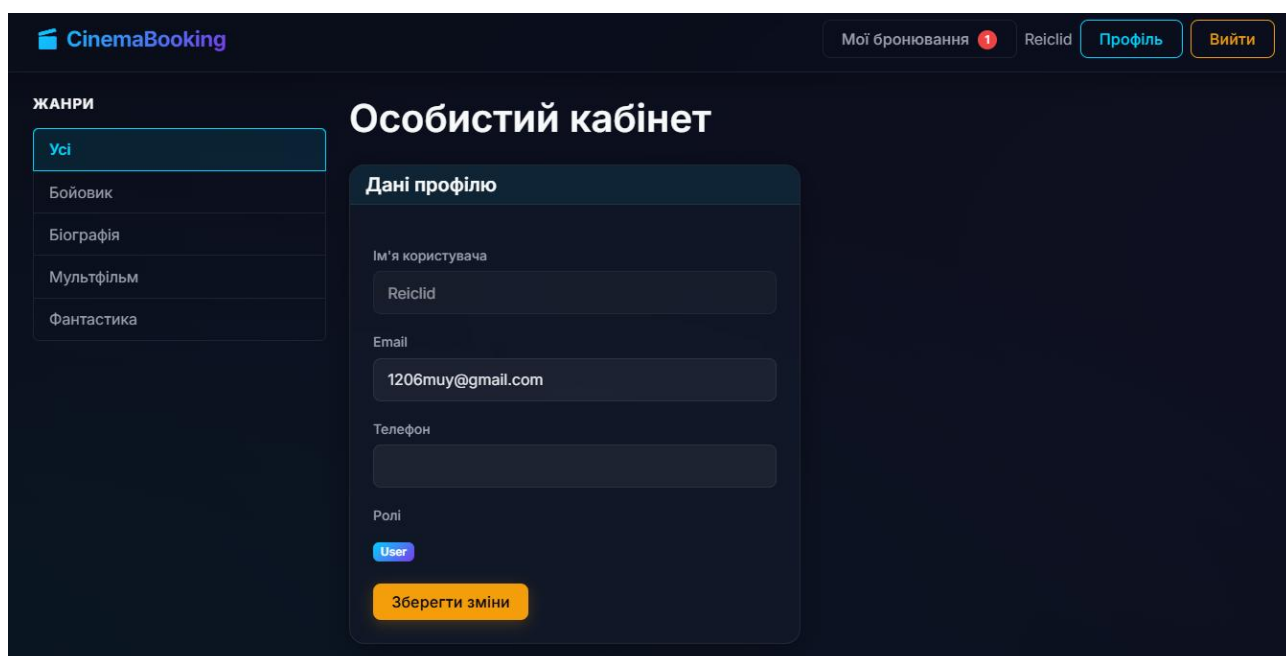


Рис. 5. Профіль користувача

Завдання 2.

В рішенні створити новий WebAPI проект. Цей проект повинен реалізовувати всі функціональності, які наявні в MVC проекті у вигляді REST-ендпоінтів. Для кращої організації і перевикористання коду слід перенести файли для роботи з БД (репозиторії, контексти, моделі) в окрему бібліотеку і додати посилання на неї в MVC та WebAPI проектах. Встановіть необхідні залежності для проекту бібліотеки. Після цього налаштуйте ConnectionString'и для створених БД для WebAPI проекту в файлі appsettings.json. Таким чином, після перенесення файлів для роботи з БД в окрему бібліотеку ми можемо використовувати всі необхідні сервіси в обох проектах.

		Грушевицький Д.І.			ДУ «Житомирська політехніка».26.121.08.000 – Лр5	Арк.
		Українець М.О.				
Змн.	Арк.	№ докум.	Підпис	Дата		7

Лістинг коду(CRUD операції кінозалу):

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using CinemaBooking.Data;
using CinemaBooking.Data.Models;

namespace CinemaBooking.Api.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    [Authorize]
    public class CinemaHallsController : ControllerBase
    {
        private ICinemaRepository repository;

        public CinemaHallsController(ICinemaRepository repo)
        {
            repository = repo;
        }

        [HttpGet]
        [AllowAnonymous]
        public IActionResult GetAll()
        {
            return Ok(repository.CinemaHalls.ToList());
        }

        [HttpGet("{id}")]
        [AllowAnonymous]
        public IActionResult Get(long id)
        {
            var hall = repository.CinemaHalls.FirstOrDefault(h => h.CinemaHallID == id);
            if (hall == null) return NotFound();
            hall.Movies = repository.Movies.Where(m => m.CinemaHallID == id).ToList();
            return Ok(hall);
        }

        [HttpPost]
        [Authorize(Roles = "Admin")]
        public IActionResult Create([FromBody] CinemaHall hall)
        {
            if (!ModelState.IsValid) return BadRequest(ModelState);
            repository.CreateCinemaHall(hall);
            return CreatedAtAction(nameof(Get), new { id = hall.CinemaHallID }, hall);
        }

        [HttpPut("{id}")]
        [Authorize(Roles = "Admin")]
        public IActionResult Update(long id, [FromBody] CinemaHall hall)
        {
            var existing = repository.CinemaHalls.FirstOrDefault(h => h.CinemaHallID == id);
            if (existing == null) return NotFound();
            existing.Name = hall.Name;
            existing.Capacity = hall.Capacity;
            existing.Location = hall.Location;
            repository.SaveCinemaHall(existing);
            return Ok(existing);
        }

        [HttpDelete("{id}")]
```

		Грушевицький Д.І.			ДУ «Житомирська політехніка».26.121.08.000 – Лр5	Арк.
		Українець М.О.				8
Змн.	Арк.	№ докум.	Підпис	Дата		


```

[Authorize(Roles = "Admin")]
public IActionResult Delete(long id)
{
    var hall = repository.CinemaHalls.FirstOrDefault(h => h.CinemaHallID == id);
    if (hall == null) return NotFound();
    repository.DeleteCinemaHall(hall);
    return NoContent();
}
}
}

```

Лістинг коду(CRUD операції фільмів):

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using CinemaBooking.Data;
using CinemaBooking.Data.Models;

namespace CinemaBooking.Api.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    [Authorize]
    public class MoviesController : ControllerBase
    {
        private ICinemaRepository repository;

        public MoviesController(ICinemaRepository repo)
        {
            repository = repo;
        }

        [HttpGet]
        [AllowAnonymous]
        public IActionResult GetAll()
        {
            return Ok(repository.Movies.ToList());
        }

        [HttpGet("{id}")]
        [AllowAnonymous]
        public IActionResult Get(long id)
        {
            var movie = repository.Movies.FirstOrDefault(m => m.MovieID == id);
            if (movie == null) return NotFound();
            return Ok(movie);
        }

        [HttpPost]
        [Authorize(Roles = "Admin")]
        public IActionResult Create([FromBody] Movie movie)
        {
            if (!ModelState.IsValid) return BadRequest(ModelState);
            repository.CreateMovie(movie);
            return CreatedAtAction(nameof(Get), new { id = movie.MovieID }, movie);
        }

        [HttpPut("{id}")]
        [Authorize(Roles = "Admin")]

```

		Грушевицький Д.І.			ДУ «Житомирська політехніка».26.121.08.000 – Лр5	Арк.
		Українець М.О.				9
Змн.	Арк.	№ докум.	Підпис	Дата		

```

public IActionResult Update(long id, [FromBody] Movie movie)
{
    var existing = repository.Movies.FirstOrDefault(m => m.MovieID == id);
    if (existing == null) return NotFound();
    existing.Title = movie.Title;
    existing.Description = movie.Description;
    existing.Genre = movie.Genre;
    existing.TicketPrice = movie.TicketPrice;
    existing.CinemaHallID = movie.CinemaHallID;
    repository.SaveMovie(existing);
    return Ok(existing);
}

[HttpDelete("{id}")]
[Authorize(Roles = "Admin")]
public IActionResult Delete(long id)
{
    var movie = repository.Movies.FirstOrDefault(m => m.MovieID == id);
    if (movie == null) return NotFound();
    repository.DeleteMovie(movie);
    return NoContent();
}
}
}
}

```

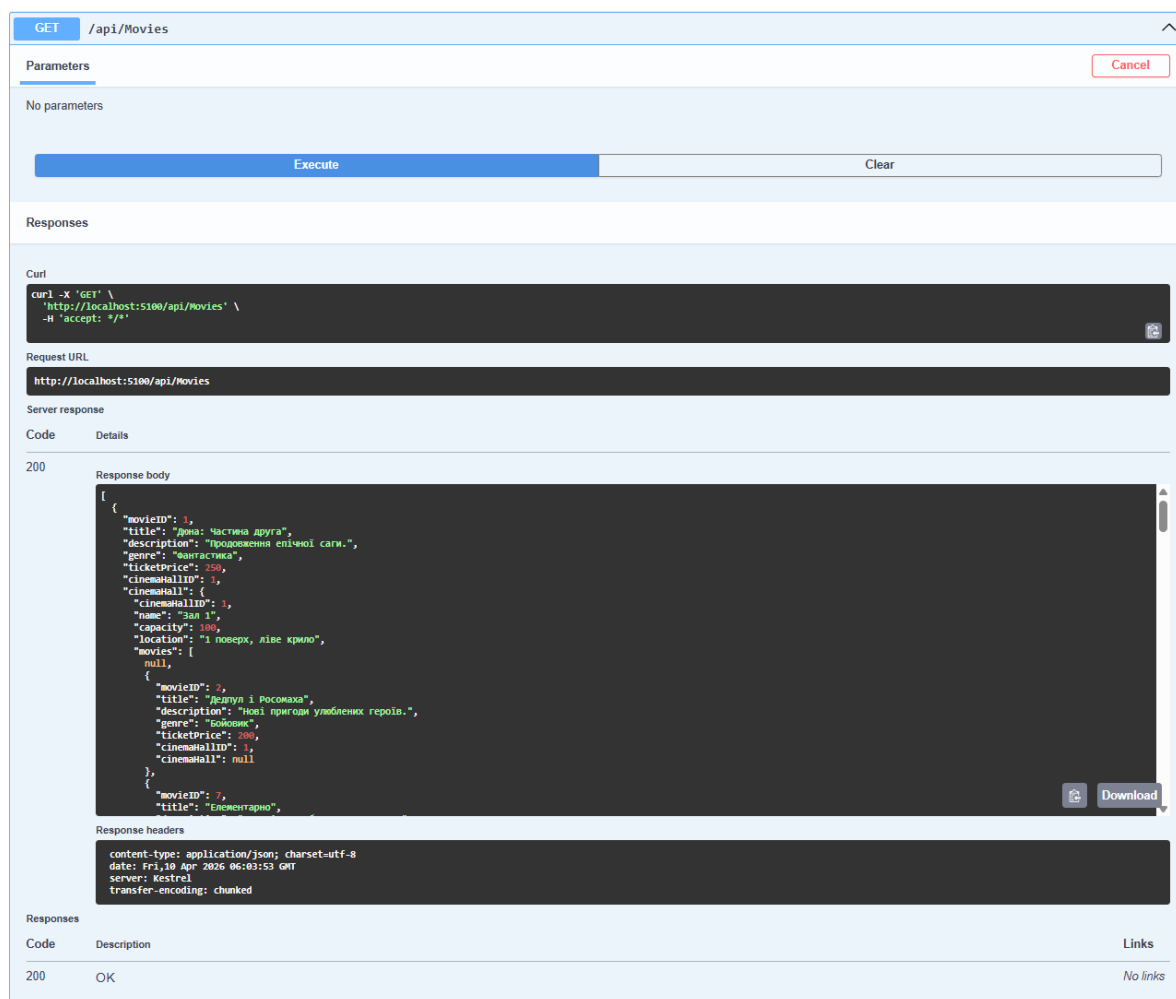


Рис. 6. Операція отримання фільмів

Завдання 3.

Реалізуйте автентифікацію та авторизацію користувача з використанням механізму JSON Web Token (JWT). Вона повинна працювати наступним чином:

1. Клієнт відправляє запит на login-ендпоінт зі своїм логіном та паролем.
2. Якщо користувач з таким логіном та паролем існує, тоді сервер повинен повернути згенерований JSON Web Token.
3. Для отримання доступу до дій, які вимагають авторизації, клієнт відправляє в запиті заголовок Authorization зі значенням Bearer TokenValue, де TokenValue – токен, який було згенеровано під час автентифікації.
4. Якщо токен валідний – сервер виконає запит, інакше поверне помилку 401 Unauthorized.

Лістинг коду(AccountController):

```
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;

namespace CinemaBooking.Api.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class AccountController : ControllerBase
    {
        private UserManager<IdentityUser> userManager;
        private RoleManager<IdentityRole> roleManager;
        private IConfiguration configuration;

        public AccountController(UserManager<IdentityUser> userMgr,
            RoleManager<IdentityRole> roleMgr,
            IConfiguration config)
        {
            userManager = userMgr;
            roleManager = roleMgr;
            configuration = config;
        }

        [HttpPost("register")]
        public async Task<ActionResult> Register([FromBody] RegisterRequest model)
        {
            var user = new IdentityUser
            {
                UserName = model.UserName,
                Email = model.Email
            };

            var result = await userManager.CreateAsync(user, model.Password);
            if (result.Succeeded)
            {
                string role = model.Role ?? "User";
                if (await roleManager.RoleExistsAsync(role))
                {
                    await userManager.AddToRoleAsync(user, role);
                }
            }
        }
    }
}
```

		Грушевицький Д.І.			ДУ «Житомирська політехніка».26.121.08.000 – Лр5	Арк.
		Українець М.О.				11
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    }
    return Ok(new { Message = "Користувача зареєстровано успішно" });
}

return BadRequest(result.Errors);
}

[HttpPost("login")]
public async Task<ActionResult> Login([FromBody] LoginRequest model)
{
    var user = await userManager.FindByNameAsync(model.UserName);
    if (user != null && await userManager.CheckPasswordAsync(user, model.Password))
    {
        var roles = await userManager.GetRolesAsync(user);

        var claims = new List<Claim>
        {
            new Claim(ClaimTypes.Name, user.UserName!),
            new Claim(ClaimTypes.NameIdentifier, user.Id),
            new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
        };

        foreach (var role in roles)
        {
            claims.Add(new Claim(ClaimTypes.Role, role));
        }

        var key = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(configuration["Jwt:Key"]!));
        var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

        var token = new JwtSecurityToken(
            issuer: configuration["Jwt:Issuer"],
            audience: configuration["Jwt:Audience"],
            claims: claims,
            expires: DateTime.Now.AddHours(3),
            signingCredentials: creds
        );

        return Ok(new
        {
            Token = new JwtSecurityTokenHandler().WriteToken(token),
            Expiration = token.ValidTo
        });
    }

    return Unauthorized(new { Message = "Невірне ім'я або пароль" });
}

public class RegisterRequest
{
    public string UserName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
    public string? Role { get; set; }
}

public class LoginRequest
{
    public string UserName { get; set; } = string.Empty;

```

		Грушевицький Д.І.			ДУ «Житомирська політехніка».26.121.08.000 – Лр5	Арк.
		Українець М.О.				12
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        public string Password { get; set; } = string.Empty;
    }
}

```

ЛІСТИНГ коду(Program):

```

using Microsoft.EntityFrameworkCore;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;
using CinemaBooking.Data;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers()
    .AddJsonOptions(opts => {
        opts.JsonSerializerOptions.ReferenceHandler =
            System.Text.Json.Serialization.ReferenceHandler.IgnoreCycles;
    });

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddDbContext<CinemaDbContext>(opts => {
    opts.UseSqlite(builder.Configuration["ConnectionStrings:CinemaConnection"]);
});

builder.Services.AddDbContext<ApplIdentityDbContext>(opts => {
    opts.UseSqlite(builder.Configuration["ConnectionStrings:IdentityConnection"]);
});

builder.Services.AddIdentity<IdentityUser, IdentityRole>(opts => {
    opts.Password.RequiredLength = 8;
    opts.Password.RequireDigit = true;
    opts.Password.RequireLowercase = true;
    opts.Password.RequireUppercase = true;
    opts.Password.RequireNonAlphanumeric = false;
    opts.User.RequireUniqueEmail = true;
}).AddEntityFrameworkStores<ApplIdentityDbContext>();

builder.Services.AddAuthentication(opts => {
    opts.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    opts.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(opts => {
    opts.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = builder.Configuration["Jwt:Issuer"],
        ValidAudience = builder.Configuration["Jwt:Audience"],
        IssuerSigningKey = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!))
    };
});

```

		Грушевицький Д.І.			ДУ «Житомирська політехніка».26.121.08.000 – Лр5	Арк.
		Українець М.О.				
Змн.	Арк.	№ докум.	Підпис	Дата		13

```
builder.Services.AddScoped<ICinemaRepository, EFCinemaRepository>();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();

SeedData.EnsurePopulated(app);
IdentitySeedData.EnsurePopulated(app).Wait();

app.Run();
```

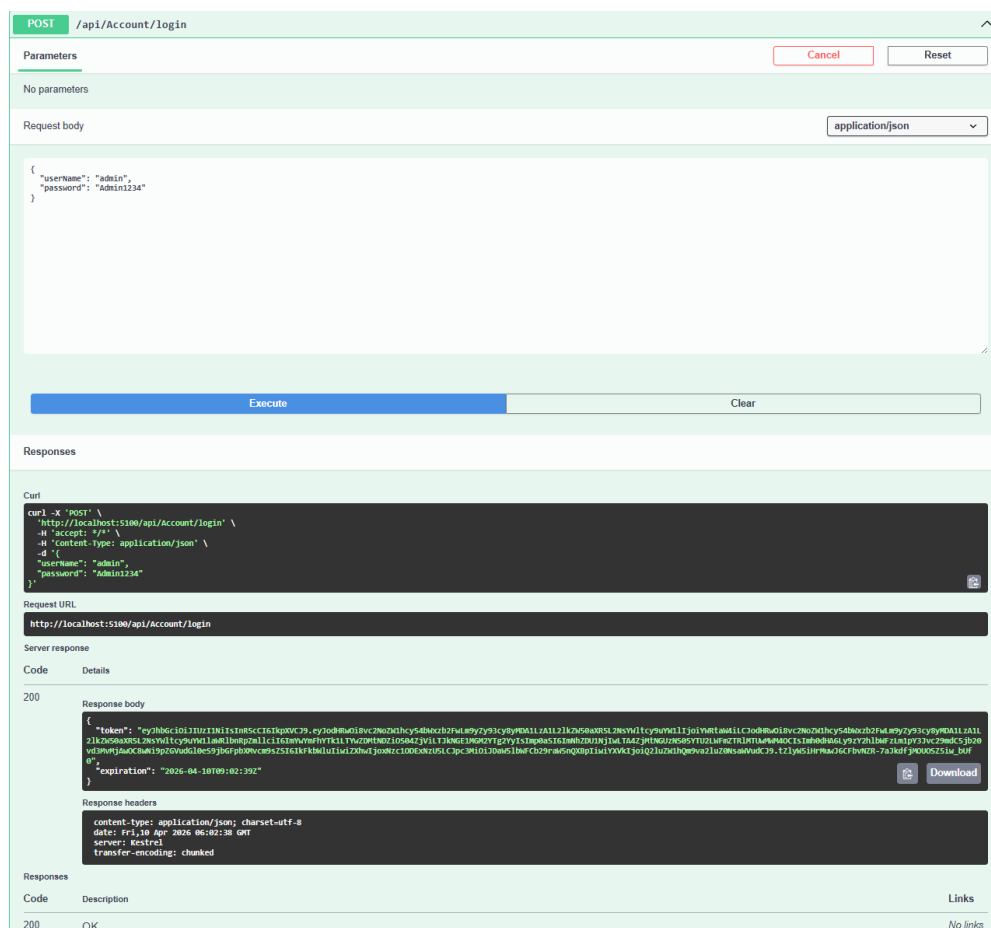


Рис. 7. Операція логіну

Висновок: в ході виконання роботи набув навичок роботи з механізмами аутентифікації та авторизації користувачів в ASP.NET, набув навичок роботи з Web API.

		Грушевицький Д.І.			ДУ «Житомирська політехніка».26.121.08.000 – Лр5	Арк.
		Українець М.О.				14
Змн.	Арк.	№ докум.	Підпис	Дата		